ASSISTANT • 设计篇

文件管理模块

从「输入文件名就能打开」，到一个分五阶段落地的文件子系统

设计理念 · 设计想法 · 模块特点

时间线

2026-08-15 第一版文件搜索

2026-08-25 市场调研与构思

2026-09-21 架构设计定稿

2026-09-24 A1 止血落地

下一阶段 搜索基建 A2–A7

CONTENTS

这份 PPT 讲四件事

01

起点

文件搜索最初是怎么做的，
当时定下了哪些取舍

02

想法

为什么它从一个搜索框，
长成一个文件子系统

03

理念

三条核心设计取向：
分层、分阶段、正确性优先

04

特点

与同类产品不同的地方，
以及能力蓝图与落地节奏

起点

项目初期的三步：文件搜索是「助手能用」的最后一块拼图

Phase 0

环境搭建

Hello 窗口跑起来
工具链打通

第 1 周

Phase 1

MVP 原型

托盘 + 全局热键 + 搜索浮窗
先让「一键呼出」成立

第 2–4 周

Phase 2

助手能用

日志 / 历史库 / 模糊匹配 /
计算器 / 应用启动器 / 文件搜索

第 5–8 周
→ v0.2.0

第一版文件搜索的目标非常朴素：输入文件名（比如 report.docx、照片）→ 模糊匹配用户目录 → 回车用默认程序打开。当时没人打算做「文件管理」，只是想让搜索框里多一路结果。

当年的四个决定，和两条主动写下的取舍

四个设计决定（2026-08-15）

① 内存索引：启动时建一次，之后每次按键只扫内存，不再递归磁盘

② 只收六个常用目录：桌面 / 文档 / 下载 / 图片 / 音乐 / 视频

③ 跳过巨目录：node_modules、__pycache__ 等

④ 上限 2 万条；文件不进默认列表（否则空查询会拖死界面）

当时就写下的两条取舍

① 启动延迟：建索引在窗口显示前同步执行，目录很大时会慢。

② 遍历浪费：QDirIterator 无法跳过子树 ——「若未来遇到大目录，可改为手写递归精确剪枝」。

为什么这些取舍值得单独记一笔

因为没有一条是「以后再说」——**每条都写清了「为什么这么选」和「什么情况下会失效」**。三个月后再回头看，这两条取舍直接变成了施工图。

当初写下的两条取舍，后来都真的发生了

当初写下的取舍	后来实测到的情况	怎么处理的
「建索引在窗口显示前同步执行，目录很大时会慢」	本机实测 0.7–2.4 秒，窗口要等索引建完才出现；桌面一个目录就有 3.5 万个文件	列入后续阶段：后台扫描 + 冷启动先可用
「QDirIterator 无法跳过子树，未来可改为手写递归精确剪枝」	一个特殊目录（指向父目录的链接）造成 126 条重复索引，路径叠了 63 层	A1 已按当初设想的方向改成手写递归 + 剪枝

好的取舍记录，就是后来的施工图。

写下取舍的人，和三个月后来修它的人，往往是同一个 —— 但那时已经记不清当初为什么了。文档里那句「若未来遇到大目录，可改为手写递归」，让 A1 不用重新做一次决策，只需要执行它。

设计想法 • 升级

想清楚一件事：文件搜索 ≠ 文件管理

过去的边界

一句话：在搜索框里多加一路结果。

文件只是 SearchResult 里的一种 type。

能搜到就够 —— 搜不到，用户自己也没办法。

现在的边界

一句话：一个独立的文件子系统。

搜索框只是它对外的一个出口，不是全部。

要同时保证两件相反的事：找得到 + 不漏掉。

由此得到最核心的一条设计理念：把「面」分开

检索面

让用户「找到想要的」 · **查询 • 打分 • 排序 • 融合**

采集面

保证「不漏掉文件」 · **扫描 • 归类 • 排除 • 追踪**

设计理念 ①

两种「过滤」必须分清 —— 这是整个设计的枢纽

采集过滤

慢 · 要扫盘

决定「什么进数据库」

- 根目录、目录级排除、工程目录识别
- 改动代价：要重新扫盘，慢
- 所以必须在界面上明确提示「正在重建索引」

检索过滤

快 · 不扫盘

决定「内存快照装什么」

- 种类白名单：只要文档 / 只要图片 / 全都要
- 改动代价：从数据库重建快照，数十毫秒
- 所以可以做成界面上一个随时可点的开关

分界线是 **files 表**：能进表的数据，都是「磁盘上真实存在」的全量结果（只受排除规则约束）；用户看到的列表，才是按白名单筛过的。

一条重要例外：下载收件箱不服从白名单。否则用户把白名单设成「只要 PDF」，刚下载的安装包就会从收件箱里消失 —— 那是数据丢失级的事故，不是筛选问题。

设计理念 ②

架构一次设计完整，功能分阶段落地

接口先定死

门面：只做「持依赖 / 转发 / 持快照 / 暴露状态」四件事

快照：查询层只能通过只读原语访问数据

Provider：A7 之后插件也挂在同一个接口上

能力慢慢加

后面的位图预筛、缓存、SoA 存储都是可选优化

加它们不改调用方 —— 因为表示藏在接口之后

所以第一阶段可以只用最朴素、最不容易错的实现

配套的三条工程习惯

① 每步都能独立编译、独立提交、独立验收；② 旧实现始终留一条可用退路（一个开关就能回退）；③ 先量基准再谈优化 —— 不做没有实测支撑的性能改造。

设计理念 ③

正确性优先于资源：宁可不给答案，也不给错答案

索引满了也不驱逐

早期想过「按使用频率淘汰」，已明确废弃 —— 被淘汰后用户搜不到自己存在的文件，
行为上等于「文件凭空消失」。改为：软预算告警、硬上限停止并提示。

去向不明就说不知道

文件被移走后，索引只知道「它走了」，不知道「它去哪了」。
多候选一律标记为未知，绝不猜一个看起来像的位置。

默认只建议，不自动动手

不自动删除任何文件；不自动移动（默认只给归档建议）；
所有移动都留日志、可撤销、同名冲突零覆盖。

为什么把它当第一原则：文件是用户的真实资产。搜索工具出错，用户只会觉得「不好用」；文件工具出错，用户会丢东西。

模块特点

和同类工具不一样在哪

纯本地 · 无外部依赖

- 不引入全文检索引擎，不依赖云服务
- 只用 Qt + SQLite + Win32，产物是一个可执行文件

查询不碰数据库

- 数据以不可变内存快照发布，读的一侧完全免锁
- 数据库只负责「冷启动加速」和「使用频率存储」

键盘优先 · 中文友好

- 输入即搜，40ms 防抖；输入法预编辑阶段不触发检索
- 后续可开启拼音首字母（默认关闭）

每一步都可退

- 新旧实现共存，一个配置项即可回退
- A1 是一个独立补丁，可单独回滚

能力蓝图

用户视角的四层能力：找得到 → 可控 → 不丢 → 会打理

找得到	文件名模糊匹配（已有）→ 路径参与、子序列、分词	v0.4.0
可控	种类过滤、工程目录排除、查询语法（只要文档 / ext:pdf）	v0.4.x
不丢	下载收件箱：下载完成自动收纳；文件移走改名也能找回	v0.4.x–v0.5.x
会打理	归档建议（可选自动）、重复文件查重、接入聊天软件来源	v0.5.x

四层之间的关系：「找得到」是地基，「可控」让结果不吵，「不丢」和「会打理」才是文件管理真正的价值。

落地节奏

五个阶段：搜索基建 → 过滤排除 → 下载中转 → 去向追踪 → 自动治理

A

搜索基建

索引 · 查询
实时更新

v0.4.0
A1 已完成

B

过滤与排除

种类过滤
工程排除

v0.4.x

C

下载中转

下载收件箱
一键归档

v0.4.x
–v0.5.x

D

去向追踪

移走改名
也能找到

v0.5.x

E

自动治理

归档建议
查重 · 多来源

v0.5.x

当前进度： A 阶段的第一步（A1 止血）已经落地并通过验收；接下来的 A2–A3 会重建索引，同时解决「文档目录搜不到」这个问题。

SUMMARY

从一个搜索框里的一路结果，到一个分五阶段落地的文件子系统。

变的	范围：从「能搜到文件名」扩到「文件不丢、可控、会打理」
不变的	那条原则：宁可不给答案，也不给错答案
沉淀下来的	每条取舍都写清「为什么」和「何时失效」，它就成了后来的施工图

依据：docs/devlog/2026-08-15.md | docs/features/市场调研与功能构思.md | docs/architecture/ 三份设计稿 | docs/devlog/2026-09-24.md